

Lab Assignment 2: VERILOG LEXICAL CONVENTIONS

Module: Verilog & Logic Synthesis

Overview:

Lexical conventions are the fundamental building blocks of Verilog syntax. This module covers how the compiler interprets whitespace, comments, identifiers, keywords, and number representation. Mastery of these rules is critical for avoiding hard-to-find synthesis and simulation errors when implementing complex logic.

Part 1: Theoretical Assignment

Objective: Evaluate understanding of Verilog's lexical rules through code analysis and correction.

Task 1: Identifier and Case Sensitivity Analysis

Determine whether the following identifiers are Valid or Invalid. If invalid, briefly explain why.

1. data_bus_32
2. \$display_enable
3. _cdc_sync_01
4. \cpu_reset_n (Note the trailing space)
5. SoC-AXI-Interface
6. wire_internal-sig
7. module
8. Clock_50MHz
9. \display_val
10. 1st_data_line
11. \reset#_n
12. \$display_val
13. DATA and data (Are these considered the same variable?)

Task 2: Number Formats and Logic Values

Translate the following descriptions into standard Verilog literal integers or real numbers.

1. A **16-bit hexadecimal** number with the value **FF0A**.
2. An **8-bit binary number** where the **upper 4 bits are unknown (x)** and the **lower 4 bits are high-impedance (z)**.
3. An **unsized decimal number** representing **4096. (two ways)**
4. A **4-bit binary** number representing **-5**.
5. An **unsized decimal** number representing **-4096. (two ways)**
6. A **real number** representing **1.25×10^{-4}** .
7. An **unsized decimal** number representing **the first 15 digits of pi (π)** after the decimal point.
8. A **4-bit octal** number representing **10 in decimal**

Task 3: Number Representation

Fill in the missing values in the table below to demonstrate your understanding of radix conversions and bit sizing.

Verilog Literal	Bit Width	Radix / Base	Binary Equivalent	Decimal Value
8'hA3				
6'h2x				
4'b101z				
12'o47				
8'd255				
'h1F				
-15				
325				
5'sd10				
3'sd12				
-6'd7				
10'bx0_101z				
5'shC				

Part 2: Lab Activity (Hands-On)

- **Objective:** Identify and **resolve intentional lexical errors** within a provided **Verilog module**.
- **Scenario:** You have received an RTL file for a simple reset synchronizer—a critical component used in Clock Domain Crossing (CDC) to safely bring an asynchronous reset into a local clock domain. However, the engineer who wrote the code made several critical lexical mistakes that will prevent compilation.
- **Task:**
 1. Review the **.v** code below.
 2. Identify **all lexical errors**.
 3. **Rewrite the module to compile successfully.**

```

Module 1st_reset_sync (
    input wire clk,
    input wire async_rst_n,
    output reg sync_rst_n
);
/* This block comment explains the module
/* The clock is active high */
and reset is active low
*/
// Internal registers for the 2-stage synchronizer

reg meta_reg;
(* preserve reg sync_reg;

// Define a delay parameter using an invalid base format
parameter DELAY = 8'd15;
parameter MAX_VAL = 4'b1021;

always @(posedge clk or negedge async_rst_n) begin
    if (!async_rst_n) begin
        Meta_reg <= 1'b0;
        Sync_reg <= 1'b0;
    end else begin
        Meta_reg <= 1'B1;
        Sync_reg <= Meta_reg;
    end
end

/* Drive the output
assign sync_rst_n = sync reg;
end module

```

Part 3: Practical Lab

Objective:

Write, compile, and simulate a Verilog testbench that actively utilizes all specified lexical conventions.

Lab Requirements:

You will create a single Verilog module named **lexical_demo** that does not require any inputs or outputs (a pure testbench). You must use a Verilog simulator (**such as ModelSim, Questasim, EDAPlayground, or Icarus Verilog**) to run this code and **view the console output**.

Your code must explicitly include and demonstrate the following:

- **Comments:** Use both **single-line** (//) and **multi-line** (/* .. */) comments to document your code.
- **Attributes:** Add a dummy **synthesis attribute** (e.g., (* keep = "true" *)) to at least one variable declaration.
- **Case Sensitivity & Identifiers:** Declare **two distinct integer variables** with the same letters but different capitalization (e.g., **Counter** and **counter**) and assign them different values.
- **Literal Integer Numbers:** Assign values using **binary** ('b'), **hexadecimal** ('h'), and **decimal** ('d') formats. Use **underscores** (_) to improve readability.
- **Logic Values:** Assign a **binary value** containing **x (unknown)** and **z (high-impedance)** to a variable.
- **Literal Real Numbers:** Declare a **real variable** and assign it a **fractional value** using **scientific notation**.
- **Strings:** Declare a **reg** wide enough to **hold a string (8 bits / character)** and **assign a greeting message to it**.
- **White Space:** Format your code cleanly using **spaces, tabs, and blank lines** to separate logical blocks.

Starter Template:

Copy this template into your simulator and complete the missing sections marked by **//TODO**.

```
/*
  Lab: Verilog Lexical Conventions
  Description: Demonstrates identifiers, numbers, strings, and attributes.
*/

// TODO: Add a module-level attribute here (e.g., (* optimize = "off" *))
module lexical_demo;
// TODO:
// 1. Strings
// A string requires 8 bits per character. "Hello World" is 11 characters.
// Declare a register named 'my_string'

// 2. Case Sensitivity & Identifiers
// Declare two 8-bit registers named 'Data_Val' and 'data_val'

// 3. Logic Values & Literal Integers
// Declare a 12-bit register named 'mixed_logic'

// 4. Literal Real Numbers
// Declare a 'real' variable named 'analog_voltage'

// 5. Attributes on variables
// use (* keep = "true" *) to a net called 'protected_wire'

  initial begin
    // --- Assignments ---
// TODO:
// String assignment
// TODO: assign "Hello World" to my_string

// TODO: Assign 8'hAA to 'Data_Val' and 8'h55 to 'data_val'

// TODO: Assign a 12-bit binary number to 'mixed_logic' (1, 0, x, z)
// Example: 12'b1010_xxxx_zzzz;

// TODO: Assign a real number to 'analog_voltage' (e.g., 3.3e-3)

// --- Display Results ---
// $display statements automatically print to the console.

    $display("--- Lexical Conventions Output ---");
    $display("String Value: %s", my_string);
// TODO: Add $display statements to print variables.
// Hint: Use %h for hex, %b for binary, and %f for real numbers.
    $finish;
  end
endmodule
```

Deliverables

Submit your **report** containing **all the answers** and your **.v files** containing all requested modules. You should simulate the modules to verify correct operation.